

8. If a sequential search method is searching for a value that is stored in the last element of a 10,000-element array, how many elements will the search code have to read to locate the value?
9. Look at the following array definition:


```
double[][] sales = new double[8][10];
```

 - a. How many rows does the array have?
 - b. How many columns does the array have?
 - c. How many elements does the array have?
 - d. Write a statement that stores a number in the last column of the last row in the array.

Programming Challenges

MyProgrammingLab™ Visit www.myprogramminglab.com to complete many of these Programming Challenges online and get instant feedback.

1. Rainfall Class

Write a `RainFall` class that stores the total rainfall for each of 12 months into an array of doubles. The program should have methods that return the following:

- the total rainfall for the year
- the average monthly rainfall
- the month with the most rain
- the month with the least rain

Demonstrate the class in a complete program.

Input Validation: Do not accept negative numbers for monthly rainfall figures.

2. Payroll Class

Write a `Payroll` class that uses the following arrays as fields:

- **employeeId.** An array of seven integers to hold employee identification numbers. The array should be initialized with the following numbers:

```
5658845 4520125 7895122 8777541
8451277 1302850 7580489
```

- **hours.** An array of seven integers to hold the number of hours worked by each employee
- **payRate.** An array of seven doubles to hold each employee's hourly pay rate
- **wages.** An array of seven doubles to hold each employee's gross wages

The class should relate the data in each array through the subscripts. For example, the number in element 0 of the `hours` array should be the number of hours worked by the employee whose identification number is stored in element 0 of the `employeeId` array. That same employee's pay rate should be stored in element 0 of the `payRate` array.

In addition to the appropriate accessor and mutator methods, the class should have a method that accepts an employee's identification number as an argument and returns the gross pay for that employee.

Demonstrate the class in a complete program that displays each employee number and asks the user to enter that employee's hours and pay rate. It should then display each employee's identification number and gross wages.

Input Validation: Do not accept negative values for hours or numbers less than 6.00 for pay rate.

3. Charge Account Validation

Create a class with a method that accepts a charge account number as its argument. The method should determine whether the number is valid by comparing it to the following list of valid charge account numbers:

5658845	4520125	7895122	8777541	8451277	1302850
8080152	4562555	5552012	5050552	7825877	1250255
1005231	6545231	3852085	7576651	7881200	4581002



VideoNote

The Charge
Account
Validation
Problem

These numbers should be stored in an array or an `ArrayList` object. Use a sequential search to locate the number passed as an argument. If the number is in the array, the method should return `true`, indicating the number is valid. If the number is not in the array, the method should return `false`, indicating the number is invalid.

Write a program that tests the class by asking the user to enter a charge account number. The program should display a message indicating whether the number is valid or invalid.

4. Charge Account Modification

Modify the charge account validation class that you wrote for Programming Challenge 3 so it reads the list of valid charge account numbers from a file. Use Notepad or another text editor to create the file.

5. Larger Than n

In a program, write a method that accepts two arguments: an array and a number n . Assume that the array contains integers. The method should display all of the numbers in the array that are greater than the number n .

6. Driver's License Exam

The local Driver's License Office has asked you to write a program that grades the written portion of the driver's license exam. The exam has 20 multiple choice questions. Here are the correct answers:

1. B	6. A	11. B	16. C
2. D	7. B	12. C	17. C
3. A	8. A	13. D	18. B
4. A	9. C	14. A	19. D
5. C	10. D	15. D	20. A

A student must correctly answer 15 of the 20 questions to pass the exam.

Write a class named `DriverExam` that holds the correct answers to the exam in an array field. The class should also have an array field that holds the student's answers. The class should have the following methods:

- `passed`. Returns `true` if the student passed the exam, or `false` if the student failed
- `totalCorrect`. Returns the total number of correctly answered questions

- `totalIncorrect`. Returns the total number of incorrectly answered questions
- `questionsMissed`. An `int` array containing the question numbers of the questions that the student missed

Demonstrate the class in a complete program that asks the user to enter a student's answers, and then displays the results returned from the `DriverExam` class's methods.

Input Validation: Only accept the letters A, B, C, or D as answers.

7. Quarterly Sales Statistics

Write a program that lets the user enter four quarterly sales figures for six divisions of a company. The figures should be stored in a two-dimensional array. Once the figures are entered, the program should display the following data for each quarter:

- A list of the sales figures by division
- Each division's increase or decrease from the previous quarter (this will not be displayed for the first quarter)
- The total sales for the quarter
- The company's increase or decrease from the previous quarter (this will not be displayed for the first quarter)
- The average sales for all divisions that quarter
- The division with the highest sales for that quarter

Input Validation: Do not accept negative numbers for sales figures.

8. Grade Book

A teacher has five students who have taken four tests. The teacher uses the following grading scale to assign a letter grade to a student, based on the average of his or her four test scores:

Test Score	Letter Grade
90–100	A
80–89	B
70–79	C
60–69	D
0–59	F

Write a class that uses a `String` array or an `ArrayList` object to hold the five students' names, an array of five characters to hold the five students' letter grades, and five arrays of four doubles each to hold each student's set of test scores. The class should have methods that return a specific student's name, the average test score, and a letter grade based on the average.

Demonstrate the class in a program that allows the user to enter each student's name and his or her four test scores. It should then display each student's average test score and letter grade.

Input Validation: Do not accept test scores less than zero or greater than 100.

9. Grade Book Modification

Modify the grade book application in Programming Challenge 8 so that it drops each student's lowest score when determining the test score averages and letter grades.

10. Lottery Application

Write a `Lottery` class that simulates a lottery. The class should have an array of five integers named `lotteryNumbers`. The constructor should use the `Random` class (from the Java API) to generate a random number in the range of 0 through 9 for each element in the array. The class should also have a method that accepts an array of five integers that represent a person's lottery picks. The method is to compare the corresponding elements in the two arrays and return the number of digits that match. For example, the following shows the `lotteryNumbers` array and the user's array with sample numbers stored in each. There are two matching digits (elements 2 and 4).

`lotteryNumbers` array:

7	4	9	1	3
---	---	---	---	---

User's array:

4	2	9	7	3
---	---	---	---	---

In addition, the class should have a method that returns a copy of the `lotteryNumbers` array.

Demonstrate the class in a program that asks the user to enter five numbers. The program should display the number of digits that match the randomly generated lottery numbers. If all of the digits match, display a message proclaiming the user a grand prize winner.

11. Array Operations

Write a program with an array that is initialized with test data. Use any primitive data type of your choice. The program should also have the following methods:

- `getTotal`. This method should accept a one-dimensional array as its argument and return the total of the values in the array.
- `getAverage`. This method should accept a one-dimensional array as its argument and return the average of the values in the array.
- `getHighest`. This method should accept a one-dimensional array as its argument and return the highest value in the array.
- `getLowest`. This method should accept a one-dimensional array as its argument and return the lowest value in the array.

Demonstrate each of the methods in the program.

12. Number Analysis Class

Write a class with a constructor that accepts a file name as its argument. Assume the file contains a series of numbers, each written on a separate line. The class should read the contents of the file into an array, and then displays the following data:

- The lowest number in the array
- The highest number in the array
- The total of the numbers in the array
- The average of the numbers in the array

This chapter's source code folder, available at www.pearsonhighered.com/gaddis, contains a text file named *Numbers.txt*. This file contains twelve random numbers. Write a program that tests the class by using this file.

13. Name Search

If you have downloaded this book's source code (the companion Web site is available at www.pearsonhighered.com/gaddis), you will find the following files in the *Chapter 07* folder:

- *GirlNames.txt* – This file contains a list of the 200 most popular names given to girls born in the United States for the years 2000 through 2009.
- *BoyNames.txt* – This file contains a list of the 200 most popular names given to boys born in the United States for the years 2000 through 2009.

Write a program that reads the contents of the two files into two separate arrays, or ArrayLists. The user should be able to enter a boy's name, a girl's name, or both, and the application will display messages indicating whether the names were among the most popular.

14. Population Data

If you have downloaded this book's source code (the companion Web site is available at www.pearsonhighered.com/gaddis), you will find a file named *USPopulation.txt* in the *Chapter 07* folder. The file contains the midyear population of the United States, in thousands, during the years 1950 through 1990. The first line in the file contains the population for 1950, the second line contains the population for 1951, and so forth.

Write a program that reads the file's contents into an array. The program should display the following data:

- The average annual change in population during the time period
- The year with the greatest increase in population during the time period
- The year with the smallest increase in population during the time period

15. World Series Champions

If you have downloaded this book's source code (the companion Web site is available at www.pearsonhighered.com/gaddis), you will find a file named *WorldSeriesWinners.txt*. This file contains a chronological list of the winning teams in the World Series from 1903 through 2009. (The first line in the file is the name of the team that won in 1903, and the last line is the name of the team that won in 2009. Note that the World Series was not played in 1904 or 1994, so those years are skipped in the file.)

Write a program that lets the user enter the name of a team, and then displays the number of times that team has won the World Series in the time period from 1903 through 2009.



TIP: Read the contents of the *WorldSeriesWinners.txt* file into an array, or an ArrayList. When the user enters the name of a team, the program should step through the array or ArrayList, counting the number of times the selected team appears.

16. 2D Array Operations

Write a program that creates a two-dimensional array initialized with test data. Use any primitive data type that you wish. The program should have the following methods:

- `getTotal`. This method should accept a two-dimensional array as its argument and return the total of all the values in the array.
- `getAverage`. This method should accept a two-dimensional array as its argument and return the average of all the values in the array.
- `getRowTotal`. This method should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The method should return the total of the values in the specified row.
- `getColumnTotal`. This method should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a column in the array. The method should return the total of the values in the specified column.
- `getHighestInRow`. This method should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The method should return the highest value in the specified row of the array.
- `getLowestInRow`. This method should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The method should return the lowest value in the specified row of the array.

Demonstrate each of the methods in this program.

17. Phone Book ArrayList

Write a class named `PhoneBookEntry` that has fields for a person's name and phone number. The class should have a constructor and appropriate accessor and mutator methods. Then write a program that creates at least five `PhoneBookEntry` objects and stores them in an `ArrayList`. Use a loop to display the contents of each object in the `ArrayList`.

18. Trivia Game

In this programming challenge, you will create a simple trivia game for two players. The program will work like this:

- Starting with player 1, each player gets a turn at answering 5 trivia questions. (There are 10 questions, 5 for each player.) When a question is displayed, four possible answers are also displayed. Only one of the answers is correct, and if the player selects the correct answer, he or she earns a point.
- After answers have been selected for all of the questions, the program displays the number of points earned by each player and declares the player with the highest number of points the winner.

You are to design a `Question` class to hold the data for a trivia question. The `Question` class should have `String` fields for the following data:

- A trivia question
- Possible answer 1
- Possible answer 2
- Possible answer 3
- Possible answer 4
- The number of the correct answer (1, 2, 3, or 4)

The `Question` class should have appropriate constructor(s), accessor, and mutator methods.

The program should create an array of 10 `Question` objects, one for each trivia question. (If you prefer, you can use an `ArrayList` instead of an array.) Make up your own trivia questions on the subject or subjects of your choice for the objects.

19. Lo Shu Magic Square

The Lo Shu Magic Square is a grid with 3 rows and 3 columns, shown in Figure 7-31. The Lo Shu Magic Square has the following properties:

- The grid contains the numbers 1 through 9 exactly.
- The sum of each row, each column, and each diagonal all add up to the same number. This is shown in Figure 7-32.

In a program you can simulate a magic square using a two-dimensional array. Write a method that accepts a two-dimensional array as an argument, and determines whether the array is a Lo Shu Magic Square. Test the function in a program.

Figure 7-31 Lo Shu Magic Square

4	9	2
3	5	7
8	1	6

Figure 7-32 Row, column, and diagonal sums in the Lo Shu Magic Square

